.NET Minimal APIs

Introduction

.NET Minimal APIs provide a lightweight and streamlined approach to building HTTP APIs in ASP.NET Core. Introduced in **.NET 6** and enhanced in **.NET 7, .NET 8, and newer versions**, Minimal APIs allow developers to create RESTful services with significantly less boilerplate code compared to traditional MVC Controllers.

Minimal APIs are ideal for:

- Microservices
- Backend services
- Internal APIs
- Cloud-native applications
- Proof of Concepts (POCs)
- Serverless applications
- High-performance APIs

Unlike the traditional Controller-based approach, Minimal APIs define endpoints directly in **Program.cs** (or extension methods), reducing the amount of code while maintaining the full power of ASP.NET Core.

Why Minimal APIs?

Before Minimal APIs, even a simple API required multiple files.

Example structure:

```
Controllers/
    ProductController.cs

Models/
    Product.cs

Program.cs

Startup.cs (.NET 5)
```

A simple GET endpoint required:

- Controller

- Route attributes
- Dependency Injection
- Startup configuration

With Minimal APIs:

```
var app = builder.Build();

app.MapGet("/", () => "Hello World");

app.Run();
```

A complete API is created with just a few lines of code.

Traditional Controller vs Minimal API

Traditional MVC

```
HTTP Request
      |
Routing
      |
Controller
      |
Service
      |
Repository
      |
Database
```

Multiple files are required.

Minimal API

```
HTTP Request
      |
MapGet()
      |
Business Logic
      |
Database
```

Less boilerplate.

Simpler architecture.

Creating a Minimal API

Create a new project.

```
dotnet new web
```

Generated Program.cs

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.MapGet("/", () =>
{
    return "Welcome to Minimal API";
});

app.Run();
```

Run:

```
dotnet run
```

Output:

```
Welcome to Minimal API
```

Basic HTTP Endpoints

Minimal APIs support all HTTP verbs.

GET

```
app.MapGet("/products", () =>
{
    return new[]
    {
        "Laptop",
        "Mouse",
        "Keyboard"
```

```
    };
});
```

POST

```
app.MapPost("/products", (Product product) =>
{
    return Results.Created($"/products/{product.Id}", product);
});
```

PUT

```
app.MapPut("/products/{id}", (int id, Product product) =>
{
    return Results.Ok(product);
});
```

DELETE

```
app.MapDelete("/products/{id}", (int id) =>
{
    return Results.Ok($"Deleted Product {id}");
});
```

Route Parameters

Route parameters are automatically bound.

```
app.MapGet("/products/{id}", (int id) =>
{
    return $"Product Id = {id}";
});
```

Request

```
GET /products/5
```

Response

```
Product Id = 5
```

Query Parameters

Example

```
app.MapGet("/search", (string keyword) =>
{
    return $"Searching {keyword}";
});
```

Request

```
/search?keyword=laptop
```

Response

```
Searching laptop
```

Request Body Binding

Minimal APIs automatically bind JSON.

Model

```
public class Product
{
    public int Id { get; set; }

    public string Name { get; set; }

    public decimal Price { get; set; }
}
```

Endpoint

```
app.MapPost("/products", (Product product) =>
{
    return Results.Ok(product);
});
```

Request JSON

```
{
  "id":1,
  "name":"Laptop",
```

```
  "price":85000
}
```

Returning Results

Minimal APIs use the **Results** helper.

Examples

```
Results.Ok()

Results.NotFound()

Results.BadRequest()

Results.Created()

Results.NoContent()

Results.Unauthorized()

Results.Problem()
```

Example

```
app.MapGet("/products/{id}", (int id) =>
{
    if(id <= 0)
        return Results.BadRequest();

    return Results.Ok(id);
});
```

Dependency Injection

Minimal APIs fully support Dependency Injection.

Service

```
public interface IMessageService
{
    string GetMessage();
}
```

```
public class MessageService : IMessageService
{
    public string GetMessage()
    {
        return "Hello from DI";
    }
}
```

Registration

```
builder.Services.AddScoped<IMessageService, MessageService>();
```

Endpoint

```
app.MapGet("/message",
(IMessageService service) =>
{
    return service.GetMessage();
});
```

The service is injected automatically.

Working with Entity Framework Core

Register DbContext.

```
builder.Services.AddDbContext<AppDbContext>();
```

Example

```
app.MapGet("/products",
(AppDbContext db) =>
{
    return db.Products.ToList();
});
```

Adding data

```
app.MapPost("/products",
(AppDbContext db, Product product) =>
{
    db.Products.Add(product);
```

```
        db.SaveChanges();

        return Results.Created($"/products/{product.Id}", product);
    });
```

Route Groups

Route Groups help organize endpoints.

Example

```
var products = app.MapGroup("/products");

products.MapGet("/", GetProducts);

products.MapGet("/{id}", GetProduct);

products.MapPost("/", CreateProduct);

products.MapDelete("/{id}", DeleteProduct);
```

Benefits

- Cleaner code
- Shared authorization
- Shared filters
- Shared versioning

Endpoint Filters (.NET 7+)

Endpoint Filters execute before or after an endpoint.

Example

```
app.MapPost("/products",
(Product product) =>
{
    return Results.Ok(product);
})
.AddEndpointFilter(async (context, next) =>
{
    Console.WriteLine("Before");
```

```
        var result = await next(context);

        Console.WriteLine("After");

        return result;
    });
```

Uses

- Validation
- Logging
- Performance monitoring
- Authorization
- Auditing

Configuration

Inject IConfiguration.

```
app.MapGet("/config",
(IConfiguration config) =>
{
    return config["AppName"];
});
```

Logging

Inject ILogger.

```
app.MapGet("/log",
(ILogger<Program> logger) =>
{
    logger.LogInformation("Endpoint called");

    return "Logged";
});
```

Authentication

Authentication middleware works exactly like MVC.

```
builder.Services.AddAuthentication();

builder.Services.AddAuthorization();
```

Middleware

```
app.UseAuthentication();

app.UseAuthorization();
```

Protect endpoint

```
app.MapGet("/admin", () =>
{
    return "Admin Area";
})
.RequireAuthorization();
```

Validation

Simple validation

```
app.MapPost("/products",
(Product product) =>
{
    if(string.IsNullOrWhiteSpace(product.Name))
        return Results.BadRequest();

    return Results.Ok(product);
});
```

Complex validation can use:

- FluentValidation
- Data Annotations
- Endpoint Filters

Error Handling

Global Exception Handler

```
app.UseExceptionHandler();
```

Custom response

```
app.MapGet("/error", () =>
{
    throw new Exception("Something went wrong");
});
```

Swagger Support

Register Swagger.

```
builder.Services.AddEndpointsApiExplorer();

builder.Services.AddSwaggerGen();
```

Middleware

```
app.UseSwagger();

app.UseSwaggerUI();
```

Open

```
https://localhost:5001/swagger
```

Producing Typed Responses

Example

```
app.MapGet("/products/{id}",
Results<Ok<Product>, NotFound> (int id) =>
{
    var product = GetProduct(id);

    return product is null
        ? TypedResults.NotFound()
        : TypedResults.Ok(product);
});
```

Benefits

- Better compile-time checking
- Better Swagger documentation

Organizing Large Minimal APIs

Instead of putting everything inside Program.cs, create extension methods.

Example

```
public static class ProductEndpoints
{
    public static void MapProductEndpoints(this WebApplication app)
    {
        app.MapGet("/products", GetProducts);

        app.MapPost("/products", CreateProduct);
    }
}
```

Program.cs

```
app.MapProductEndpoints();
```

This keeps the project clean and maintainable.

Advantages of Minimal APIs

- Less boilerplate code
- Faster development
- High performance
- Built-in Dependency Injection
- Supports middleware
- Supports authentication and authorization
- Easy to understand
- Ideal for microservices
- Native Swagger support
- Excellent for cloud-native applications

Limitations

Minimal APIs are not suitable for every scenario.

Limitations include:

- Large applications can become difficult to organize if endpoints are not modularized.
- Attribute-based routing is replaced with fluent endpoint mapping.
- Teams familiar with MVC may require time to adapt.
- Complex UI-driven applications often benefit more from Controllers and Views.

Best Practices

- Keep endpoints small and focused.
- Move business logic into services instead of writing it directly in endpoint delegates.
- Use Dependency Injection for all external dependencies.
- Group related endpoints using `MapGroup()`.
- Use Endpoint Filters for cross-cutting concerns such as logging and validation.
- Enable Swagger for API documentation.
- Apply authentication and authorization consistently.
- Return appropriate HTTP status codes using `Results` or `TypedResults`.
- Organize large APIs using extension methods or dedicated endpoint classes.
- Follow RESTful naming conventions.

Minimal API Architecture

```
HTTP Request
      |
Middleware Pipeline
      |
Route Matching
      |
Minimal API Endpoint
      |
Business Service
      |
Repository
      |
```

```
      Entity Framework Core
            |
      SQL Server
```

This architecture demonstrates that Minimal APIs are not limited to simple applications. They integrate seamlessly with Dependency Injection, Entity Framework Core, authentication, middleware, logging, and other ASP.NET Core features.

Minimal API vs MVC Controller

Feature	**Minimal API**	**MVC Controller**
Boilerplate	Very Low	High
Learning Curve	Easy	Moderate
Performance	Excellent	Excellent
Dependency Injection	Yes	Yes
Swagger Support	Yes	Yes
Middleware Support	Yes	Yes
Authentication	Yes	Yes

Authorization	Yes	Yes
Best For	Microservices, Lightweight APIs	Enterprise Applications, Large Projects
File Structure	Minimal	Multiple Controllers and Files

Real-World Example

Consider an inventory management service for a retail application.

```
Client Application
        |
        v
Minimal API Endpoint
        |
        v
Inventory Service
        |
        v
Repository
        |
        v
SQL Server Database
```

The API exposes endpoints such as:

- `GET /inventory`
- `GET /inventory/{id}`
- `POST /inventory`
- `PUT /inventory/{id}`
- `DELETE /inventory/{id}`

Business logic resides in services, while data access is handled by repositories and Entity Framework Core. This separation keeps the API clean, testable, and maintainable while leveraging the simplicity of Minimal APIs.

Summary

.NET Minimal APIs provide a modern, lightweight approach to building RESTful services with minimal boilerplate. They support all core ASP.NET Core features, including Dependency Injection, middleware, authentication, authorization, Entity Framework Core, logging, configuration, and Swagger. By reducing ceremony while maintaining flexibility, Minimal APIs are particularly well suited for microservices, cloud-native applications, internal services, and high-performance APIs. When combined with proper architectural practices such as service layers, dependency injection, and endpoint organization, Minimal APIs offer a scalable and maintainable solution for modern .NET application development.